

CENTRO UNIVERSITÁRIO UNISENAI
CURSO DE GRADUAÇÃO TECNOLÓGICA EM ANÁLISE E
DESENVOLVIMENTO DE SISTEMAS

SolarVision

Qualidade e Testes de Software

Estratégia, casos automatizados, execução e evidências

Equipe: Daniel Tavares dos Anjos, João Victor Bagatim,
Luiz Gustavo Mella Massing e Pedro Henrique de Carvalho Ferreira

FLORIANÓPOLIS
09/2026

Sumário

1. Objetivo e diagnóstico	3
2. Estratégia e critérios de qualidade	4
3. Casos automatizados e rastreabilidade	5
4. Roteiro funcional integrado — executado e aprovado	7
5. Execução e evidências	8
6. Automação e reprodução	9
7. Entregáveis e conclusão	10

1. Objetivo e diagnóstico

Definir e aplicar uma estratégia de qualidade orientada aos riscos de autenticação, consistência dos cadastros e apresentação das leituras solares. O diagnóstico identificou 17 testes existentes em quatro classes do backend, ausência de testes no frontend e ausência de integração contínua. Foram adicionados 21 casos no backend e 21 no frontend, totalizando 59 execuções automatizadas previstas na suíte final.

2. Estratégia e critérios de qualidade

Risco / atributo	Prioridade	Técnica e controle
Login indevido ou cadastro duplicado	Alta	Regressão de AuthService; cenários positivos e negativos
Limpeza associada a placa inexistente	Alta	Serviço com Mockito; confirmar que não há gravação
Entrada inválida na API	Alta	MockMvc, Bean Validation e resposta HTTP
Período invertido e agrupamento incorreto	Alta	Partições válidas/inválidas e limites de datas no dashboard
Potência exibida com escala incorreta	Alta	Valores imediatamente antes e no limiar de W/kW/MW/GW
Regressão de cadastro e alertas	Alta	Preservação e execução dos testes existentes
Nomes com acentos e hífen	Média	Testes de Unicode e entradas vazias
Build quebrado	Alta	Compilação do backend, build Vite e workflow em cada push/PR

A base da estratégia são testes unitários rápidos e determinísticos. Acima dela, testes de contrato MVC verificam entrada, validação e status HTTP. A validação integrada executada em Docker complementa essa base com PostgreSQL real, filtros JWT, transporte gRPC e navegador. Mocks isolam dependências e não comprovam persistência, autorização ou integração ponta a ponta; por isso os cenários integrados foram executados separadamente.

Entrada: fontes disponíveis, Node 24, dependências do lockfile, Maven 3.9 e JDK compatível. Dados fictícios e datas fixas; nenhum teste usa dados pessoais reais ou o banco de produção.

Saída da etapa automatizada: todos os casos executados, zero falhas/erros/ignorados e build do frontend aprovado. Os fluxos funcionais críticos foram executados no ambiente integrado Docker com Java 25, PostgreSQL e navegador. Não foi medida cobertura de linhas; a aprovação dos casos não equivale a cobertura total nem à homologação do produto.

Responsabilidades propostas: desenvolvedor mantém casos e corrige defeitos; revisor confere requisito, evidência e regressão; responsável pelo produto valida fluxos e decide publicação.

3. Casos automatizados e rastreabilidade

Pré-condição comum: dependências instaladas. Em testes de serviço, os repositórios são simulados e reinicializados a cada caso. Cada variação parametrizada conta como uma execução.

ID / arquivo	Procedimento e entrada	Resultado esperado	Quantidade
BE-AUTH / AuthServiceTest (existente)	Registrar usuário; duplicar e-mail; autenticar com credenciais válidas, usuário ausente e senha errada	Token e usuário normalizado no sucesso; exceções e ausência de token no erro	5
BE-GROUP / GroupServiceTest (existente)	Executar os cenários de grupos já presentes no código	Manter regras e verificações da suíte original	3
BE-PANEL / PanelServiceTest (existente)	Executar os cenários de placas já presentes no código	Manter regras e verificações da suíte original	4
BE-GRPC / GrpcOperationsServiceTest (existente)	Executar regras de verificação e alerta com dependências simuladas	Manter contrato do serviço; e-mail continua simulado	5
BE-CLN-01 / CleaningServiceTest	Criar limpeza para placa 1 com observação Inspeção	ID 2, placa 1, data preservada e observação sem espaços externos	1
BE-CLN-02	Criar para placa 9 ausente	NotFoundException; nenhuma gravação	1
BE-CLN-03	Consultar limpeza 9 ausente	NotFoundException	1
BE-CLN-04	Atualizar limpeza 2 para placa 3 e observação nula	Placa e data atualizadas; observação anterior removida	1
BE-CLN-05/06	Excluir registro existente e ausente	Exclusão somente no existente; erro no ausente	2
BE-DASH-01/02 / DashboardServiceTest	Data inicial maior que final; granularidade year	BadRequestException sem consulta ao repositório	2
BE-DASH-03	hora, dia, semana, mes, DIA; datas inicial e final iguais	Tokens hour/day/week/month corretos; intervalo igual aceito	5
BE-DASH-04	Bucket 01/09/2026 e potência 1500.50; granularidade nula	Fuso -03:00, valor preservado e agrupamento diário	1
BE-DASH-05	Consultar resumo sem leituras/limpezas	Total zero, zero placas e última limpeza nula	1
API-CLN-01 / CleaningControllerTest	POST /api/cleanings com {}	HTTP 400 e erros por campo; serviço não chamado	1
API-CLN-02	POST com observação de 1001 caracteres	HTTP 400; serviço não chamado	1
API-CLN-03	POST válido com observação de 0 e 1000 caracteres	HTTP 200 e encaminhamento do DTO ao serviço	2

ID / arquivo	Procedimento e entrada	Resultado esperado	Quantidade
API-CLN-04/05	GET ausente; DELETE existente	HTTP 404 com status no corpo; HTTP 204 sem corpo	2
FE-POT / power.test.js	0, 999, 1000, 999999, 1000000, 1e9, -1500	Unidade e divisor corretos, inclusive por magnitude negativa	7
FE-FMT / power.test.js	0, 1500, 12500, 125000, string numérica, null e texto inválido	Formatação pt-BR e fallback zero	7
FE-FMT-COMUM	500 W em série com unidade kW	0,50 kW	1
FE-NOME / text.test.js	Acentos, hífen, espaços, vazio, null e undefined	Capitalização Unicode e fallback vazio	6

O teste MVC usa configuração standalone: executa binding, validação, controller e handler real, com serviço simulado. Não carrega Spring Security. O cenário de POST válido verifica encaminhamento; não comprova gravação real.

4. Roteiro funcional integrado — executado e aprovado

Ambiente executado: Docker Compose isolado (solarvision-qa), PostgreSQL 17, backend Spring Boot compilado em Java 25, frontend Nginx e Chrome headless 152. Foram usados usuário, grupos, placas, leituras e limpezas sintéticos. Os seis cenários foram aprovados e as evidências estão em docs/evidencias/integracao/.

Pré-condição: ambiente descartável com PostgreSQL, backend e frontend ativos, usuário de teste e placas/grupos conhecidos. Usar dados sintéticos; registrar versão, navegador, data, esperado/obtido e captura de tela ou resposta HTTP.

ID	Passos	Resultado esperado	Estado
FT-01	Registrar e-mail novo, sair, entrar com a mesma senha	Conta criada e acesso ao dashboard	Aprovado
FT-02	Sem token, chamar GET /api/cleanings; abrir rota privada	API recusa com 401/403 e UI exige login	Aprovado
FT-03	Criar grupo e placa, atualizar e consultar novamente via API	Associação e alterações persistidas	Aprovado
FT-04	Criar limpeza para placa existente, recarregar a tela, excluir via API	Registro persistido e posterior ausência	Aprovado
FT-05	Inserir leituras conhecidas no banco de teste; filtrar datas e granularidade	Valores coerentes com fixture e fuso definido	Aprovado
FT-06	Acionar verificação/alerta pela ponte REST-gRPC	Resposta coerente e alerta persistido; sem presumir e-mail real	Aprovado

Esses casos não entram na contagem dos 59 testes automatizados. A execução confirmou migrations, consultas SQL reais e experiência em navegador. Não foram realizadas medições de carga, auditoria completa de acessibilidade ou teste de envio real de e-mail.

5. Execução e evidências

Resultado consolidado: 59 testes aprovados (38 no backend e 21 no frontend), com zero falhas. 42 testes adicionados e 17 preservados.

Frontend: Node 24.14.1; npm ci --no-audit --no-fund, npm test e npm run build. Backend: Maven 3.9.11 em container maven:3.9.11-eclipse-temurin-25; mvn -B clean verify. A execução utilizou Java 25, a mesma versão configurada no projeto.

Resultado: 38 testes de backend e 21 de frontend aprovados, zero falhas, erros ou ignorados. Os seis casos integrados também foram aprovados. Build Vite concluído com 129 módulos transformados. Os resultados consolidados estão em evidencias/resultado.json, os nomes/status dos casos em evidencias/backend-casos.json, evidencias/frontend.tap e evidencias/integracao/resultado.json. Os relatórios Surefire completos são gerados em backend/target/surefire-reports (ignorados pelo Git). O bundle de gráficos tem 577,98 kB sem compressão; avaliar otimização em trabalho posterior.

Limitações do ambiente: o Maven Wrapper do repositório está incompleto; a execução usa Maven no container. A primeira tentativa de build do Vite foi bloqueada por permissões do ambiente; houve nova execução autorizada. Os testes não apontaram defeito funcional nos cenários executados; não foram alteradas regras de produção.

6. Automação e reprodução

```
cd frontend
npm ci --no-audit --no-fund
npm test
npm run build
cd ../backend
cd ..
docker compose -f compose.qa.yml --profile test run --rm backend-tests
```

O arquivo `.github/workflows/quality.yml` executa backend com Java 25 e frontend com Node 24 em push e pull request e publica os relatórios como artefatos. O workflow foi criado; a execução local equivalente no Java 25 foi aprovada.

Para cada defeito futuro, registrar ID, severidade, pré-condições, passos mínimos, resultado esperado/obtido, evidência e teste de regressão. Corrigir, repetir o teste que falhou e executar a suíte afetada. Bloqueios de ambiente devem ser registrados como bloqueios, nunca como aprovação.

7. Entregáveis e conclusão

Foram concluídas a definição da estratégia de qualidade, a implementação de 42 novos testes automatizados, a execução da suíte de 59 testes com aprovação de todos os casos, a execução e aprovação dos 6 roteiros funcionais integrados e a construção do frontend. Também foram entregues o workflow de qualidade, as evidências de execução, o resumo no README e este relatório PDF.